

Machine Learning I

Ricardo Figueiredo up202105430

Sérgio Cardoso up202107918

May 26, 2025

Executive Summary

- **Goals:** use a classification algorithm taught on classes to evaluate its performance on a set of benchmark datasets.
- **Outline of the Approach:** base our implementation on open-source code, then modify it in order to make it more robust to the data characteristic in question.
- **Summary of Results:**
 - Tables comparing Accuracy/F1-Score/Precision/Recall/ROC across datasets
 - Comparison of the original and modified algorithms
 - Summary of the improvement of the algorithm (modified vs original)

Executive Summary

- **Goals:** use a classification algorithm taught on classes to evaluate its performance on a set of benchmark datasets.
- **Outline of the Approach:** base our implementation on open-source code, then modify it in order to make it more robust to the data characteristic in question.
- **Summary of Results:**
 - Tables comparing Accuracy/F1-Score/Precision/Recall/ROC across datasets
 - Comparison of the original and modified algorithms
 - Summary of the improvement of the algorithm (modified vs original)

- **Goals:** use a classification algorithm taught on classes to evaluate its performance on a set of benchmark datasets.
- **Outline of the Approach:** base our implementation on open-source code, then modify it in order to make it more robust to the data characteristic in question.
- **Summary of Results:**
 - Tables comparing Accuracy/F1-Score/Precision/Recall/ROC across datasets
 - Comparison of the original and modified algorithms
 - Summary of the improvement of the algorithm (modified vs original)

For this assignment, we chose the CART algorithm due to its easy learning curve. CART stands for Classification And Regression Tree which, in itself, uses a Decision Tree. Decision Trees are among the most popular models in ML, as they are used not only by CART, but also by other algorithms, such as ID3 (Iterative Dichotomiser 3) or C4.5, the latter being an extension of CART. CART uses the Gini index as its impurity measure: given a data set $D = \{\langle x_i, y_i \rangle\}_{i=1}^N$ (with N examples), $Gini(D) = 1 - \sum_{i=1}^c p_i^2$, where p_i is the probability of class i .

Decision trees, when working with imbalanced datasets, emphasize the majority class (the one with more examples), which can lead to high misclassification rates for the minority classes.

There are some ways to prevent this from happening, such as:

- Cost-Sensitive Learning (assign higher misclassification costs to the minority class)
- Sampling Techniques (balance the dataset by oversampling the minority class (using SMOTE) or undersampling the majority class (Tomek Links, ENN - Edited Nearest Neighbors))
- Alternative Evaluation Metrics to Accuracy (e.g. Precision, Recall or F1-Score)

We were given the three following sets of benchmark datasets:

- ① Noise or Outliers
- ② Class Imbalance in Binary Classification
- ③ Multiclass Classification

Since CART uses Binary Partitioning, we ended up using the set of binary classification datasets (Class Imbalance in Binary Classification). We also used this set because of the fact that CART can deal with class imbalanced data, if we apply some of the techniques mentioned before.

Experimental Setup:

- Datasets: unbalanced binary classification datasets.
- Hyperparameters of the Algorithm:
 - Maximum Depth of the Tree
 - Criterion (Gini)
 - Class Weight

Performance Estimation Methodology:

Since most of the datasets tested for this assignment are so large, we used the holdout method, in which we split the data in two sub-sets: one for training and the other for testing.

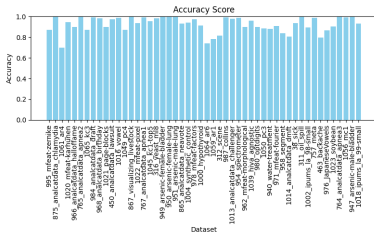
Experimental Setup:

- Datasets: unbalanced binary classification datasets.
- Hyperparameters of the Algorithm:
 - Maximum Depth of the Tree
 - Criterion (Gini)
 - Class Weight

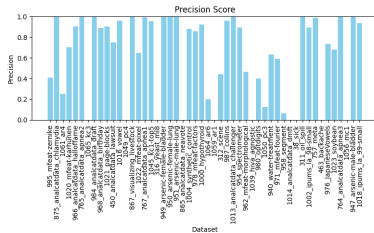
Performance Estimation Methodology:

Since most of the datasets tested for this assignment are so large, we used the holdout method, in which we split the data in two sub-sets: one for training and the other for testing.

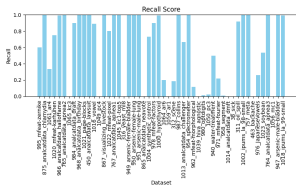
Empirical Study - Performance of the original CART algorithm



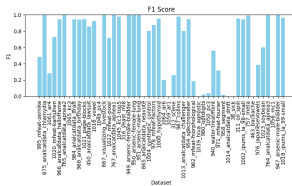
(a) Accuracy



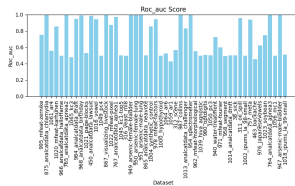
(b) Precision



(c) Recall

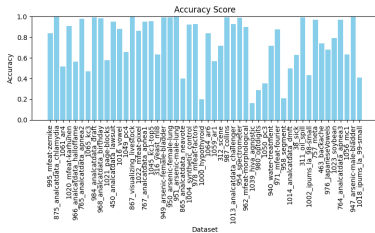


(d) F1-Score

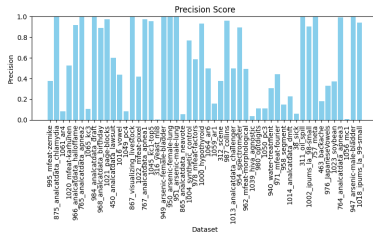


(e) ROC

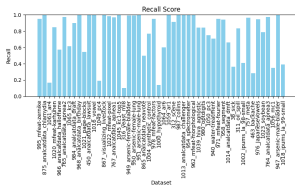
Empirical Study - Performance of the modified CART algorithm



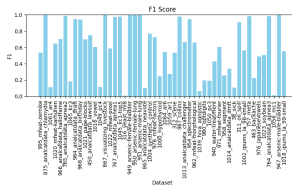
(a) Accuracy



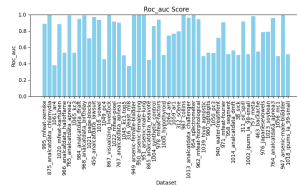
(b) Precision



(c) Recall

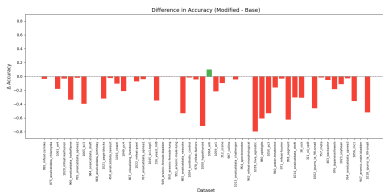


(d) F1-Score

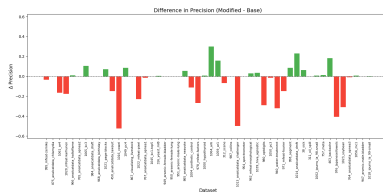


(e) ROC

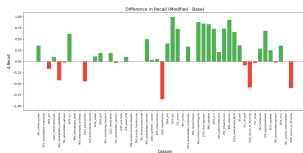
Empirical Study - Performance Comparison



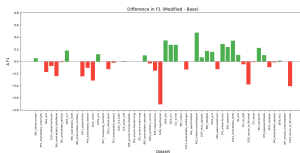
(a) Accuracy



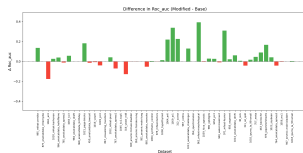
(b) Precision



(c) Recall

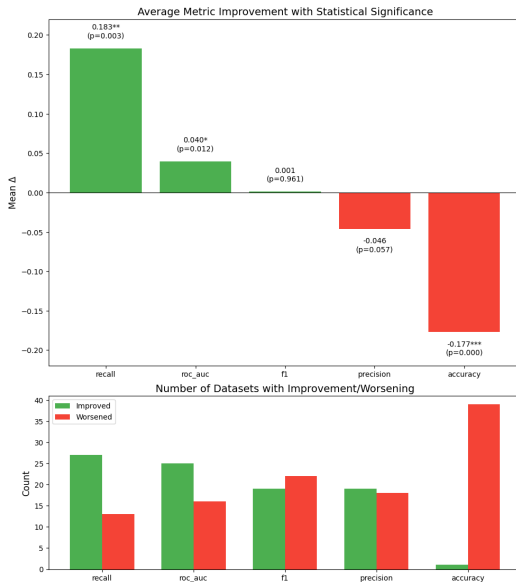


(d) F1-Score



(e) ROC

Empirical Study - Improvement Summary



Visual shorthand for p-value ranges: *** = $p < 0.001$ (most significant) | ** = $p < 0.01$ | * = $p < 0.05$ | (no stars) = Not statistically significant

Key Findings:

- Modified CART Improves Minority-Class Performance:
 - +4% ROC-AUC
 - +18.3% Recall
- Trade-offs Exist:
 - Accuracy decreased 17.7%
 - Gains were dataset-dependent

Limitations:

- Manual tuning of class weights required for each dataset.
- Noisy minority-class samples degrade performance.

Future Work:

- Automate weight tuning via grid search.

Key Findings:

- Modified CART Improves Minority-Class Performance:
 - +4% ROC-AUC
 - +18.3% Recall
- Trade-offs Exist:
 - Accuracy decreased 17.7%
 - Gains were dataset-dependent

Limitations:

- Manual tuning of class weights required for each dataset.
- Noisy minority-class samples degrade performance.

Future Work:

- Automate weight tuning via grid search.

Key Findings:

- Modified CART Improves Minority-Class Performance:
 - +4% ROC-AUC
 - +18.3% Recall
- Trade-offs Exist:
 - Accuracy decreased 17.7%
 - Gains were dataset-dependent

Limitations:

- Manual tuning of class weights required for each dataset.
- Noisy minority-class samples degrade performance.

Future Work:

- Automate weight tuning via grid search.

- **CART Implementation (Base Code Inspiration):**
<https://github.com/zziz/cart>
- **Original CART Algorithm:** Breiman, L., Friedman, J., Stone, C. J., & Olshen, R. (1984). *Classification and Regression Trees*. Chapman & Hall/CRC.
- **Evaluation Metrics & Tools:** Hunter, J. D. (2007). *Matplotlib: A 2D Graphics Environment*. IEEE.